

Código de test de la P1 2026

formato aritméticas:

ADD RD, RS, RT

formato lw, sw, beq:

LW RT, INM(RS)

SW RT, INM(RS)

BEQ RS, RT

Contenido Memoria Datos: [256, 1, 8, Nint (0 al empezar), 12, 0x00000AB0, 0, 0x0BAD0C0D, 0, 0...]

Restricciones: El valor cero se utiliza sin cargar de memoria aprovechando que es el valor inicial de todos los registros. Inicialmente se usa con el registro 0. Pero después se le asigna otro valor al registro 0 para probar si se detectan algunas dependencias. A partir de ahí se usa el registro 6 para el 0 porque a R6 no se le asigna nunca otro valor en el código.

Pseudo-código

```
const int sp_init = 256;
const int one = 1;
const int eight = 8;
volatile static int Nint = 0;
const int doce = 12;
const int valor_abort = 0x00000AB0;
int salida;
void INI(){ver código}

void RTI (void) __irq { //__irq indica que es una excepción y que hay que
    Nint++;             // que generar el prólogo y epílogo adecuado
    Print(Nint);} // Escribe un entero en el registro de salida

void Abort (void) {print(0xAB0);}
void UNDEF (void) {print(sp_init); while(1){ }}// sp_init= 0x0100
```

Escrituras en BR y MEM:

R0 = 8, 1;

R1 = 8, [Abort: 0xAB0, dec:2736], [RTI: 8,1, 8, restaurar] , 0xBAD0ADD0 (genera un abort) [Abort: 0xAB0 dec:2736], 0x80 (del jal), 0x84 (2º jal), 0xBAD0C0D0 (undef)

R2 = 16, INT1: 0, 1, 16,1, 8, 64, 256

R3 = 12, 0xBAD0ADD0, 2, 4

R4 = 32, 256

R5 = 44

R31 = 256, 264, 256 (sp)

Mem(12)= 1 (Nº INT atendidas)

Mem(24) = 44

Mem(64*4) = ABO (pila)

Mem(65*4) = 16 (pila)

IO_output : 8, 12, 256, 0, 0xAB0, 1, 0, 0xAB0, 0xBAD0C0D0 (undef)

. No a todos les llega la IRQ en el mismo momento (depende de si gestionan bien las paradas)

Valores finales contadores:

Excepciones: **4** = 1 irq+ 2(aborts) + 1 (undef))

Paradas de datos: **15** (16 si para en el jal uso)

Paradas de control: **14** (antes del bucle infinito final) = salto a ini + salto abort+ RTE + salto IRQ + RTE + BEQ + JAL+ salto abort + RTE + JAL + RET + JAL + RET + salto UNDEF

Paradas estructurales: Deberían ser **10**, pero en mi temporización salen **14**, porque la primera mac se ejecuta 3 veces (se mata las dos primeras)

Ciclos: la última escritura en IO_output se realiza en el **ciclo 117**

Reset	@0x0	10210003	beq R1, R1, INI;	Se salta siempre a la @16 donde empieza el programa (@0x8) . Parada control.
IRQ	@0x4	1021003E	beq R1, R1, RTI;	Se salta siempre a la @64*4. Parada control
DAbort	@0x8	1021005D	beq R1, R1, RT_Abort;	Se salta siempre a la @96*4. Parada control
UNDEF	@0xC	1021006C	beq R1, R1, RT_UNDEF;	Se salta siempre a la @112*4. Parada control
INI:	@0x10	081F0000	LW R31, 0(R0)	R31=Mem(0) = 256; R31 es el puntero de pila (SP)
	@0x14	08010008	LW R1, 8(R0)	R1=Mem(8) = 8;
	@0x18	0CC17004	sw r1, 7004(r6)	IO output <= R1=8; 1 ciclo LW USO
	@0x1C	04211000	ADD R2,R1, R1	Dependencia doble en R1 generada por Rt: R2= 16 (10h)
	@0x20	08030010	LW R3, 16(R0)	R3=Mem(16)=12
	@0x24	04422000	ADD R4, R2, R2	Dependencia doble en R2 generada por Rd: corto de busW (ALUout MEM). R4= 32 (20h)
	@0x28	04832800	ADD R5, R4, R3	Dependencia en R4 generada por Rd: corto ALU_out MEM; Dependencia en R3 generada por Rt: corto busW (MDR). R5= 44 =2Ch
	@0x2C	0C050018	SW R5, 24(R0)	Dependencia en R5 generada por Rd: corto ALUout. Mem(24) = 44(2Ch)
	@0x30	10050017	beq R0, R5, Err1;	Dependencia en R5 generada por Rs. Hay que parar un ciclo BEQ R0 tiene que valer 0 y R5 44, -> no se salta
	@0x34	04000000	add r0, r0, r0	para comprobar si se para aunque no haga falta
	@0x38	08200000	lw r0, 0(r1)	si no gestiona bien las dependencias hará una parada. Escribe 8 en r0
	@0x3C	00000000	nop	si no gestiona bien las dependencias hará una parada
	@0x40	08C00004	lw r0, 4(r6)	R0 =M[4]=1
	@0x44	10030013	beq R0, R3, Err2;	Dependencia en rt. Dos ciclos de parada R0 == 1 y R3 == 12 por tanto no se salta
	@0x48	0CC37004	sw r3, 7004(r6)	IO output <= R3=12;
	@0x4C	08C40000	LW R4, 0(R6)	R4=Mem(0) = 256;
	@0x50	0CC47004	sw r4, 7004(r6)	IO output <=256; 1 ciclo LW USO
	@0x54	08C30003	LW R3, 3(R6)	Abort por acceso no alineado. Se escribe x"BAD0ADD0" en R3. La RTE volverá a la instrucción siguiente
	@0x58	04001005	MAC ini R2, R0 ,R0	R2=ACC=1*1=1; 2 paradas estructurales
	@0x5C	04421804	MAC R3,R2,R2	R3=ACC=1*1+ACC=2. 2 paradas estructurales. Dependencia con R2 de MAC ini, activación de corto M->E1
	@0x60	04431804	MAC R3,R2,R3	R3=ACC=1*2+2=4. 2 paradas estructurales. Dependencia con R3 de MAC 0x5C, activación de corto M->E1
	@0x64	08C20008	LW R2, 8(R6)	R2 = Mem(8)=8
	@0x68	04421005	MAC ini R2,R2,R2	R2=ACC=8*8=64. 2 paradas estructurales. 1 parada lw-uso
	@0x6C	04431005	MAC_ini R2,R2,R3	R2=ACC=64*4=256. 2 paradas estructurales. Dependencia con R2 de MAC ini 0x68, activación de corto M->E1
	@0x70	10440001	beq R2, R4, Seguir	256=256. Se debería saltar; Riesgo datos BEQ. Dos ciclos de parada.. Parada de control
	@0x74	10000005	beq R0, R0, Err3	Si se ejecuta es un error (killIF)
Seguir:	@0x78	08C15DC0	LW R1, 24000(R6)	Abort por fuera de rango. Se escribe x"BAD0ADD0" en R1. La RTE volverá a la instrucción siguiente
	@0x7C	14010006	jal R1,6	Saltamos a la función Funct1. No debe parar por lw_uso R1 =x80. Parada de control
	@0x80	14010006	jal R1,6	Saltamos a la función Funct2. No debe parar por lw_uso. R1 =x84. Parada de control
	@0x84	FFFFFFFF		Salta la excepción UNDEF y se queda en un bucle infinito
	@0x88	0CC37004	sw r3, 7004(r6)	IO output <= R3=12; No debería ejecutarse
Err3:	@0x8C	1000FFFF	beq R0, R0, Err3	Si llegas aquí no has gestionado bien el fetch inc
Err1:	@0x90	1000FFFF	beq R0, R0, Err1	Si llegas aquí has hecho mal el salto beq R0, R5, Err1;
Err2:	@0x94	1000FFFF	beq R0, R0, Err2	Si llegas aquí has hecho mal el salto beq R0, R3, Err2;
Funct1:	@0x98	18200000	RET R1	No hace nada más que volver. Hay que parar un ciclo por riesgo de datos en R1
Funct2:	@0x9C	04260800	ADD R1,R1,R6	Sumamos 0 a R1, así que no cambia. Dependencia con R1 de JAL 0x80, activación de corto W->E
	@0xA0	18200000	RET R1	Retorna a la Instrucción siguiente. Hay que parar dos ciclos por riesgo de datos en R1
RTI:	@0x100	0FE10000	SW R1, 0(R31)	Guardamos el contenido de R1 en pila.
		0FE20004	SW R2, 4(R31)	Guardamos el contenido de R2 en pila
		08C10008	LW R1, 8(R6)	R1=Mem(8) = 8;
		07E1F800	ADD r31, R1, R31	R31=R31 +8= 264; Incrementamos el SP. Parada lw-uso

		08C2000C	LW R2, C(R6)	$R2 = Mem(C)$ En esta posición de memoria contabilizamos el número de int (Nint)
		08C10004	LW R1, 4(R6)	$R1 = Mem(4) = 1$
		04221000	ADD R2, R1, R2	$R2 = Nint++$; Parada lw-uso
		0CC27004	sw r2, 7004(r6)	$IO_output \leq R2$;
		0CC2000C	SW R2, C(R6)	$Mem(C) = Nint++$
		0CC17008	sw r1, 7008(r6)	$INT_ACK \leq 1$; (dura un ciclo)
		08C10008	LW R1, 8(R6)	$R1 = Mem(8) = 8$;
		07E1F801	SUB r31, R31, R1	$SP = SP - 8 = 256$, Parada lw-uso
		0BE10000	LW R1, 0(R31)	Restauramos el contenido de R1 de pila
		0BE20004	LW R2, 4(R31)	Restauramos el contenido de R2 de pila
		20000000	rte	Se vuelve a la instrucción que se interrumpió. Parada control retorno
		08C17FFF	LW R1, 24000(R6)	Abort por fuera de rango. Como esta instrucción se invalida no debería generar excepción
RT_Abort	@0x180	08C10014	LW R1, 20(R6)	$R1 = Mem(20) = 0x00000AB0$;
	@0x184	0CC67004	sw r6, 7004(r6)	$IO_output \leq 0$;
	@0x188	0CC17004	sw r1, 7004(r6)	$IO_output \leq 0x00000AB0$; Dependencia con R1 de la LW 0x180, activación de corto W->E
	@0x18C	20000000	rte	Se vuelve a la instrucción que se interrumpió. Parada control retorno
RT_UNDEF	@0x1C0	08C1001C	LW R1, 24(R6)	$R1 = Mem(28) = 0x0BAD0C0D$;
		0CC17004	sw r1, 7004(r6)	$IO_output \leq 0x00000AB0$; Parada lw-uso
	bucleU	1000FFFF	beq R0, R0, bucleU	